

Lecture 2 Revision Notes — Spring Boot First App

1. What We're Doing

Writing our first Spring Boot app. Goal: see how fast Spring Boot works, not understand everything yet.

2. What We're Building

Browser sends request → Spring Boot app sends response. One endpoint:

`localhost:8080/hello`.

3. What is a Port?

IP address identifies the machine (laptop). Port identifies which app on that machine (Chrome, Zoom, Spring Boot etc.) should get the data.

4. IP vs Port

IP = which machine. Port = which app on that machine. Example: `192.168.1.10:8080`.

5. What is Localhost?

`localhost` = your own machine = `127.0.0.1`. So `localhost:8080` = "my own machine, port 8080."

6. Browser & Default Ports

`http` → default port 80. `https` → default port 443. That's why we don't write port in normal URLs. Local Spring Boot apps run on 8080, so we write it manually.

7. Development Setup

Use IntelliJ IDEA (best for Java/Spring Boot). Make sure Java is installed (`java -version`) before starting.

8. Why Spring Initializr?

Creating a Spring project from scratch needs lots of repetitive setup. Spring Initializr auto-generates the project skeleton (type, Java version, dependencies, etc.).

9. What is a Dependency?

An external library your project needs (e.g., Spring Web, MySQL Connector, Lombok). Added in `pom.xml` in Maven projects.

10. Spring Boot Versions

- **SNAPSHOT** = work-in-progress, may have bugs.
- **RC** = Release Candidate, almost final.
- **Stable** = tested, use this for real projects.

11. JAR Packaging

JAR = Java Archive. Contains app code + setup needed to run — that's why Spring Boot apps run easily without external server deployment.

12. Project Structure

- `src/main/java` → your Java code
- `src/main/resources` → config files
- `pom.xml` → Maven config (dependencies)
- `application.properties` → app config (e.g., change port)
- Main class → has `main()` method, starts the app via `SpringApplication.run()`

13. What is a Controller?

Entry point for web requests — like a "receptionist" that receives requests and decides the response.

14. @RestController

Annotation that tells Spring: "this class handles HTTP requests and returns data directly."

15. Creating an Endpoint

```
java

@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

`@GetMapping("/hello")` = run this method when GET request comes to `/hello`.

16. What Happens on Run

Spring Boot auto-starts an **embedded Tomcat server** (no manual install needed) — Tomcat actually listens on the port.

17. Changing Port

In `application.properties`: `server.port=9090` → app now runs on `localhost:9090`.

18. Complete Request Flow

Browser → Tomcat → Spring MVC checks mappings → matches `/hello` → runs `sayHello()` → returns "Hello World" → back to browser.

19. Why It Feels Fast

Very little code written, but Spring Boot does a lot behind the scenes: starts context, Tomcat, Spring MVC, detects controller, registers endpoint.

20. Why Learn Internals?

Spring Boot = speed. But real debugging (404 errors, slow startup, controller not receiving requests) needs understanding of internals — Tomcat, Servlets, Spring Core, MVC, DI.

21. Industry Reality

AI tools can write `@RestController` code easily. Companies pay engineers who **understand why things break** and how to debug/design systems — not just who can write annotations.

22. Final Takeaway

Spring Boot gives speed, Spring fundamentals give control. Upcoming topics: Maven, Servlets, Tomcat, Spring Core, IoC, DI, Beans, Spring MVC, Auto-configuration.